CARAT: Class-Aware Robust Aggregation via Hidden Task Certificates for Poisoning-Resilient Federated Learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Federated learning servers must aggregate client updates without observing client data, making them vulnerable to both untargeted model poisoning and targeted backdoor attacks. Existing robust aggregators mainly rely on update geometry, coordinate-wise robust statistics, or alignment with a single trusted reference direction. These signals can be brittle when benign clients are heterogeneous and adaptive attackers remain close to the benign update manifold. We present **CARAT**, a class-aware robust aggregation rule based on *hidden task certificates*. CARAT uses server-visible labeled reference data to instantiate multiple hidden, class-balanced micro-tasks and scores each client update by the loss decrease it induces on those tasks. It then combines robust norm clipping, common-radius evaluation, worst-task optimization through a soft minimum over certificate scores, coordinate-rank anomaly penalties, and capped-simplex weight optimization with temporal regularization. This design favors updates that are broadly useful across semantic sub-tasks while suppressing clients that remain structurally extreme in rank space. In a verified FedAvg/CIFAR100/FangAttack run, CARAT substantially improves over Mean and assigns zero weight to the attacker coalition in late rounds. Because the currently available runs disable test-time evaluation, this evidence should be read as an initial empirical validation rather than a complete benchmark; nevertheless, it supports the promise of task-aware aggregation and motivates a full clean-accuracy and ASR study.

1 Introduction

Federated learning (FL) enables many clients to optimize a shared model without centralizing their raw data [1]. This decentralization is attractive for privacy, governance, and communication reasons, but it also makes the server fundamentally dependent on updates that it cannot directly verify. Poisoning is therefore a systems-level concern. Malicious clients can send updates that derail convergence, bias the learned decision boundary, or implant triggers that activate attacker-chosen behavior at inference time [2–6]. In practice, FL security is often limited by the quality of the aggregation rule.

The difficulty is not only that attackers can be strong, but also that benign clients are not identical. Under non-i.i.d. data partitions, benign updates differ in direction, norm, and sparsity for legitimate reasons. This creates a persistent ambiguity for the server: a client may look unusual because it is malicious, or simply because its local data distribution is different. Many existing defenses struggle because they rely on signals that conflate these two cases. Pairwise distances can mistake benign heterogeneity for adversarial deviation; coordinate-wise trimming can ignore cross-coordinate structure; and single-reference trust scores can miss task-specific failures.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

This observation motivates the central question behind CARAT: *can the server judge a client update by testing whether it is useful across hidden semantic sub-tasks, rather than only by asking whether it resembles other updates?* We answer this question with **CARAT**, *Class-Aware Robust Aggregation via Hidden Task Certificates*. CARAT assumes the server has access to labeled reference data from which it constructs a balanced probe pool. Rather than compressing this trusted information into one reference gradient, the server repeatedly samples hidden, class-balanced micro-tasks and evaluates each client update by the loss decrease it produces on those tasks. The resulting certificate matrix exposes a failure mode missed by geometry-only rules: an update may appear globally plausible while still degrading a subset of classes or semantic regions.

CARAT combines this task-aware signal with two additional mechanisms. First, it uses robust norm clipping and common-radius evaluation to prevent trivial scale advantages from dominating hidden-task comparisons. Second, it computes a coordinate-rank anomaly penalty that captures whether a client remains structurally extreme across many coordinates even after clipping. Aggregation weights are then optimized on a capped simplex using a worst-task softmin objective and temporal regularization toward the previous round's weights.

The resulting method is more expressive than conventional geometry-only aggregation while remaining compatible with a standard synchronous FL runtime. Conceptually, CARAT aims to separate *benign diversity* from *malicious inconsistency*: benign clients may differ, but useful updates should still help a range of hidden tasks; malicious clients may imitate benign norms or directions, but it is harder for them to remain simultaneously task-useful and rank-typical across rounds.

Contributions. Our contributions are as follows.

- We introduce hidden-task certificates as a task-aware server-side utility signal for robust FL aggregation, using multiple class-balanced micro-tasks rather than a single trusted direction.
- We formulate CARAT as a capped-simplex weight optimization problem that combines worst-task utility, rank-space anomaly suppression, and temporal regularization.
- We make explicit the implementation assumptions enforced in code, most importantly the server-side access to a labeled training dataset from which the probe pool is sampled.
- We report the current empirical evidence available in the repository and distinguish it from the full clean-test and ASR benchmark that remains to be executed.

2 Method

2.1 Problem setting

We consider synchronous FL with n clients and a global parameter vector $w_r \in \mathbb{R}^d$ at round r. Each client returns a message u_i^r. In the FLPoison runtime, CARAT first converts all client messages into a common vector-form delta representation $\Delta_i^r \in \mathbb{R}^d$ using the shared `prepare_updates` utility: for FedAvg, Δ_i^r is the difference between the client model vector and the current global model; for FedSGD and FedOpt, Δ_i^r is the vector update already produced by the client. The server then aggregates $\{\Delta_i^r\}_{i=1}^n$ into a single global step. A fraction ρ of clients may be malicious and may coordinate to craft adversarial updates.

Server knowledge. The current implementation assumes more than a tiny external probe set. At initialization, the server receives the full server-side `train_dataset` object and samples a balanced probe pool from it. CARAT then constructs hidden probe tasks only from that sampled pool, but the method still assumes server access to labeled training data. In its present form, CARAT is therefore not a data-free robust aggregation rule.

Goal. The server aims to learn aggregation weights that preserve benign diversity, prevent a malicious coalition from dominating the global step, and remain robust under both attack and benign non-i.i.d. variability.

2.2 CARAT overview

CARAT is built around three design principles.

- **Task-aware validation.** If an update is genuinely useful, it should improve more than one hidden slice of the task rather than only a narrow subset of classes.
- **Worst-case robustness.** A robust aggregator should care about the weakest hidden tasks, not only average probe improvement.
- **Structure-aware anomaly detection.** Even after norm clipping, malicious updates may remain extreme in coordinate ordering; rank-space signals complement task-aware certificates.

Round-level procedure. At communication round r, CARAT performs the following operations.

1. Robustly clip the client updates to suppress obvious norm inflation.
2. Rescale the clipped updates to a common evaluation radius so that hidden-task comparison is not dominated by raw magnitude.
3. Sample hidden, class-balanced probe tasks from the trusted probe pool.
4. Evaluate each candidate update on every hidden task and form a task-by-client certificate matrix.
5. Compute coordinate-rank anomaly penalties from random coordinate subsets.
6. Optimize aggregation weights on a capped simplex using worst-task utility, anomaly suppression, and temporal regularization.
7. Aggregate the *clipped* client updates with the optimized weights and apply the resulting server step.

The implementation initializes the temporal prior with a uniform weight vector and reuses the previous round's solution as the starting point for the next round. CARAT does not use a single transformed representation for every subproblem: the normalized updates are used only for *fair scoring* on hidden tasks, whereas the clipped updates are used for the final model step. The method thus separates *evaluation geometry* from *update geometry*.

2.3 Hidden-task and anomaly scoring

CARAT begins by sampling a balanced probe pool from the server-visible training dataset. At each round, it constructs T hidden tasks by drawing a small number of examples per class from that pool. In the current implementation, the default configuration uses a probe pool of 512 examples, $T = 8$ hidden tasks, and 4 examples per class within each task when available. If a class has fewer than the requested examples, sampling falls back to replacement; if balanced task construction fails entirely, the implementation falls back to a single random subset from the probe pool.

This class-balanced design is the source of CARAT's class-aware behavior. A malicious update may still appear reasonable in aggregate if it primarily harms minority or attacker-targeted classes; balanced hidden tasks make such failures more visible to the server.

Clipping and common-radius evaluation. Before task evaluation, CARAT robustly clips client updates. Let $\|\Delta_i\|_2$ denote the norm of client i's update. The default implementation computes a clipping threshold τ_r with a MAD-based rule:

$$\tau_r = \mathrm{median}(\{\|\Delta_i\|_2\}) + k \cdot 1.4826 \cdot \mathrm{MAD}(\{\|\Delta_i\|_2\}), \tag{1}$$

with percentile-based alternatives also available in the configuration. The clipped update is

$$\bar{\Delta}_i = \min\left(1, \frac{\tau_r}{\|\Delta_i\|_2 + \varepsilon}\right) \Delta_i. \tag{2}$$

CARAT then rescales each clipped update to a common evaluation radius q_r:

$$\tilde{\Delta}_i = q_r \frac{\bar{\Delta}_i}{\|\bar{\Delta}_i\|_2 + \varepsilon}, \tag{3}$$

where q_r is chosen as a quantile of the positive clipped norms. This step removes pure norm advantages when the server compares candidate updates on hidden tasks. Importantly, the normalized vectors are used only for evaluation; the final aggregation is computed on the clipped updates $\bar{\Delta}_i$. Appendix Proposition A.3 formalizes the key invariance induced by this normalization step.

Hidden-task certificates. For each hidden task t, let $\ell_t(w)$ denote the average cross-entropy loss of model w on that task. CARAT defines a *hidden-task certificate*

$$c_{i,t} = \ell_t(w_r) - \ell_t(w_r + \tilde{\Delta}_i). \tag{4}$$

Here, "certificate" denotes an internal task-level utility score rather than a formal certified-robustness guarantee. Positive values indicate that client i improves task t relative to the current global model.

Operationally, CARAT loads the current global weights into a server-side probe model, evaluates the baseline hidden-task losses once, and then reevaluates the same tasks after applying each candidate update $w_r + \tilde{\Delta}_i^r$. Stacking these values yields a certificate matrix $C \in \mathbb{R}^{n \times T}$. Because different tasks may have different baselines and scales, CARAT robustly standardizes each column using the column median and MAD, preserving within-task ordering while making columns comparable.

Rank-space anomaly penalties. Hidden-task utility alone is insufficient, because an attacker may overfit to a subset of probe tasks while remaining structurally abnormal. CARAT therefore augments certificate scores with a coordinate-rank anomaly signal.

For a sampled coordinate set $S \subseteq \{1, \ldots, d\}$, CARAT ranks clients coordinate-wise and measures the average normalized distance from the center rank:

$$r_i^{(S)} = \frac{1}{|S|} \sum_{j \in S} \left| \frac{\mathrm{rank}_{i,j}}{n-1} - \frac{1}{2} \right|. \tag{5}$$

The implementation repeats this procedure over several random coordinate subsets and takes the median across subsamples. A client receives a large anomaly score if its update remains persistently extreme across many coordinates. The raw anomaly scores are then robustly standardized, and only positive deviations are retained as penalties. The penalty is asymmetric: suspiciously extreme clients are downweighted, while unusually central clients are not specially rewarded.

2.4 Weight optimization and aggregation

CARAT chooses aggregation weights $\alpha \in \mathbb{R}^n$ on the capped simplex

$$\mathcal{A}(\hat{\rho}) = \left\{ \alpha \in \mathbb{R}^n : \alpha_i \geq 0,\ \sum_{i=1}^{n} \alpha_i = 1,\ \alpha_i \leq \frac{1}{(1-\hat{\rho})n} \right\}, \tag{6}$$

where $\hat{\rho}$ is an estimate of the attacker fraction. This cap prevents any single client from receiving too much mass and ensures that at least $(1-\hat{\rho})n$ clients must contribute nontrivially to the aggregate. Appendix Proposition A.1 states the corresponding feasibility and support guarantees.

Let $r \in \mathbb{R}^n$ denote the positive rank penalties and let α^{prev} be the previous round's weights. CARAT solves

$$\max_{\alpha \in \mathcal{A}(\hat{\rho})} \ \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\mathrm{prev}}\|_2^2, \tag{7}$$

where

$$\operatorname*{softmin}_{\beta}(s_1, \ldots, s_T) = -\frac{1}{\beta} \log \left(\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \right). \tag{8}$$

The first term rewards weight allocations that improve the weakest hidden tasks. The second penalizes structurally extreme clients. The third stabilizes weights across rounds and reduces unnecessary oscillation. Appendix Proposition A.2 records the basic bounds and gradient form of the softmin term used by the optimizer.

When the runtime knows the number of attackers, the code resolves $\hat{\rho}$ directly from `num_adv`; otherwise it uses a configured default. Eq. (7) also yields an interpretable selection rule. If a client performs poorly on even a few hidden tasks, the softmin term increases pressure to downweight that client. If a client remains extreme in rank space, the penalty term further decreases its mass. A client must therefore satisfy *both* a semantic criterion and a structural criterion to retain influence. Appendix Proposition A.4 further clarifies the role of the temporal prior among utility-equivalent feasible weights.

Final aggregation and complexity. After optimization, CARAT aggregates the clipped updates:

$$\Delta_{\text{agg}} = \sum_{i=1}^{n} \alpha_i \bar{\Delta}_i. \tag{9}$$

This yields the final server step.

Relative to standard aggregation, the main additional cost comes from evaluating n candidate updates over T hidden tasks and computing rank penalties on coordinate subsamples. Appendix Proposition A.5 summarizes the resulting per-round complexity. The implementation therefore exposes explicit controls for the probe pool size, the number of tasks, the number of sampled coordinates, the number of rank subsamples, and the number of projected-gradient steps, making the overhead tunable.

3 Experiments

3.1 Experimental setup

CARAT is implemented as a server aggregator in FLPoison and can be used with FedAvg, FedSGD, and FedOpt because the runtime converts all client messages into vector-form deltas before defense logic. The reproduced setup we can currently verify end-to-end uses the committed `FedAvg_CIFAR100_Resnet18.yaml` base config: 300 rounds, 20 clients, local epochs 4, learning rate 0.05, batch size 64, and a ResNet-18 model on CIFAR100. The same base config sets `evaluate=false`, so the available CARAT logs do *not* contain clean-test accuracy, clean-test loss, ASR, or ASR loss.

The benchmark infrastructure is broader than the current verified CARAT run. The committed `cifar100.yaml` batch spec already sweeps i.i.d. and Dirichlet non-i.i.d. partitions with $\alpha \in \{0.1, 0.5, 1.0\}$, attacker fractions $\{0.2, 0.3, 0.4\}$, untargeted attacks {NoAttack, MinMax, MinSum, ALIE, FangAttack}, and baselines {Mean, TriGuardFL, FLDetector, FLTrust, MultiKrum, NormClipping}. CARAT is implemented and logs are available for an i.i.d. FangAttack run, but CARAT has not yet been inserted into the committed batch spec. We therefore separate *benchmark scope supported by code* from *empirical evidence already executed for CARAT*.

When evaluation is enabled, the coordinator records clean test accuracy and clean test loss every round, and for data-poisoning or hybrid attacks it additionally records ASR and ASR loss. The default CARAT hyperparameters in the defense preset are: probe pool size 512, 8 hidden tasks, 4 probe samples per class, probe batch size 128, probe-radius quantile 0.5, MAD clipping with $k = 2.5$, default attacker prior $\hat{\rho} = 0.2$, certificate temperature $\beta = 12.0$, rank weight 0.20, prior weight 0.05, 4096 sampled coordinates, 5 rank subsamples, 60 optimization steps, and optimizer learning rate 0.5.

4 Results

4.1 Current empirical results

Table 1 summarizes the strongest evidence currently verified end-to-end in the repository: single-seed FedAvg runs on CIFAR100 with an i.i.d. split, 20 clients, 20% attackers (4 malicious client slots), ResNet-18, 300 rounds, and FangAttack. The available logs cover Mean, TriGuardFL, and CARAT under the same base training configuration. Because that configuration sets `evaluate=false`, the table reports *training-only* diagnostics and runtime rather than clean-test accuracy or ASR.

CARAT substantially improves over simple averaging in the verified run. Relative to Mean, it increases round-299 training accuracy by about 5.9 points and reduces training loss by about 0.24. At the same time, the current evidence does *not* support a broader claim of uniform dominance across strong baselines: on this single training-only run, TriGuardFL finishes with slightly higher training accuracy, whereas CARAT attains the lowest training loss. We therefore interpret Table 1 as evidence that the method is functioning as intended, not as a complete comparative benchmark.

The most diagnostic CARAT-specific signal is the behavior of the learned weights. In this benchmark, the coordinator assigns the first `num_adv` worker slots to attackers, so clients 0–3 are malicious. Late

Defense	Train acc. @299	Train loss @299	Mean round time (s)	Total runtime
Mean	0.5699	1.5187	1.66	8m25s
TriGuardFL	**0.6385**	1.2932	7.37	37m07s
CARAT	0.6286	**1.2748**	9.49	47m37s

Table 1: Current empirical evidence from repository logs for FedAvg on CIFAR100 under i.i.d. FangAttack. These are training-only diagnostics because the reproduced runs disable test-time evaluation.

Factor	Directly supported config keys
Probe construction	`num_probe_pool`, `num_probe_tasks`, `probe_samples_per_class`
Clipping / normalization	`clip_method`, `clip_k`, `clip_percentile`, `probe_radius_quantile`
Objective shaping	`certificate_beta`, `rank_weight`, `prior_weight`, `rho_hat`
Rank estimation	`rank_num_coords`, `rank_num_subsamples`
Optimizer budget	`optimizer_steps`, `optimizer_lr`

Table 2: Ablation axes already exposed by the current CARAT implementation. This is the safest starting point for a code-aligned ablation section.

in training, CARAT repeatedly logs weight vectors of the form

$$\alpha = [0, 0, 0, 0, 0.0625, 0.0625, \ldots, 0.0625],$$

which means the optimizer places zero mass on the entire malicious coalition while distributing the remaining mass uniformly over benign clients. At epoch 298, for example, the log records task scores $[0.2879, 0.2784, 0.2859, 0.2781, 0.2533, 0.3009, 0.2930, 0.2876]$, a softmin value of 0.2820, evaluation radius 18.1484, and clip norm 18.3227. These quantities indicate that the implementation is doing more than hard filtering: it is identifying weak hidden tasks under a common scoring radius and then applying the optimized weights to the clipped updates used for aggregation.

These observations are encouraging, but they do not yet establish improved clean-test accuracy, reduced ASR, or robustness across non-i.i.d. regimes. A complete evaluation still requires reruns with `evaluate=true` and inclusion of CARAT in the batch benchmark spec.

5 Ablation Experiments

The cleanest ablations are those already supported by the current configuration interface. The first ablation package should isolate:

- probe pool size,
- number of hidden tasks,
- per-class probe samples,
- clipping rule, clipping threshold, and evaluation-radius quantile,
- rank penalty weight λ_r,
- prior weight λ_p,
- softmin temperature β,
- rank subsampling budget and optimizer steps.

These ablations are directly supported by the preset groups summarized in Table 2. This design already supports a concrete methodological question: how much of CARAT's behavior comes from task construction, clipping and normalization, rank penalties, or temporal regularization?

Two additional structural ablations would strengthen the final paper but require small code patches rather than simple config changes: replacing the softmin objective with average utility, and bypassing hidden-task certificates entirely. The current implementation does not expose those switches as top-level parameters, so the paper should not imply that those ablations have already been run.

The final empirical narrative should first establish robustness with clean-test metrics, then evaluate backdoor settings using both ASR and clean utility, and finally use the supported ablations above to determine whether CARAT's gains come primarily from hidden-task validation, rank-space filtering, or their interaction.

6 Limitations

The most important limitation is the server-side data assumption. In the current code path, CARAT does not receive an externally curated probe pool; instead, it receives the server-visible training dataset and samples a balanced probe pool from it. This is a materially stronger assumption than fully data-free robust aggregation.

A second limitation is computational cost. Compared with lightweight rules such as Mean, CARAT adds repeated hidden-task forward passes, rank subsampling, and projected optimization. On the current FangAttack/CIFAR100 run, the mean round time is 9.49s for CARAT versus 1.66s for Mean and 7.37s for TriGuardFL. Any final empirical case must therefore justify CARAT's extra server-side cost with corresponding clean-test or ASR gains, not only with training-time diagnostics.

A third limitation is empirical coverage. The strongest verified CARAT evidence is still a single-seed, training-only FangAttack run; the committed batch spec does not yet include CARAT, and the base configuration disables evaluation entirely. This means the current manuscript supports a methodology claim and an initial empirical result, but not yet a full benchmark claim.

A fourth limitation is adaptive security. An informed adversary could try to approximate the probe distribution, optimize against the clipping rule, or mimic benign rank patterns. CARAT is designed to raise the difficulty of such attacks by combining heterogeneous signals, but it does not eliminate the adaptive-attack problem. A stronger final version of the paper should therefore include at least one explicitly adaptive attack study.

Broader impact. Robust aggregation methods can improve the reliability of federated learning in settings where model failures carry real cost, including medical, financial, and safety-sensitive applications. At the same time, stronger defenses may shift attacker incentives rather than remove them. For this reason, the broader impact of CARAT lies not only in the method itself, but also in the benchmark and evaluation protocol through which it is stress-tested. Clear reporting of assumptions, ablations, and failure cases is therefore part of the contribution.

7 Related Work

Prior work on poisoning-resilient federated learning can be organized by the signal used to decide whether a client should influence the global model. At a high level, the literature relies on geometric centrality, coordinate-wise robustness, trusted reference data, historical behavior, or explicit backdoor diagnostics. This organization is useful for situating CARAT because the method changes the *unit of validation* from client-to-client similarity to client-to-task utility.

Geometry-based Byzantine robustness. Geometry-based defenses seek updates that remain central under pairwise distances. Krum and its extensions are canonical examples [7, 8]. Their appeal lies in a clear robustness intuition: malicious clients should look isolated from the benign majority. The main limitation is that this intuition weakens under client heterogeneity. When benign updates are naturally dispersed, geometric centrality is no longer a reliable proxy for benignness.

Coordinate-wise robust statistics. Coordinate-wise median and trimmed mean replace or discard extreme coordinates independently [9]. These methods are computationally simple and often surprisingly strong, but they intentionally ignore interactions across coordinates. As a result, they may miss attacks that preserve per-coordinate plausibility while corrupting higher-order structure in the update.

Heterogeneity-aware robust aggregation. The interaction between Byzantine robustness and non-i.i.d. data has become an important theme in its own right. In particular, prior work shows that many robust aggregators degrade sharply under heterogeneity and proposes bucketing as a simple way to adapt robust rules to non-i.i.d. settings [10]. This line of work is directly relevant to CARAT

because it starts from the same failure mode but addresses it from a different angle: instead of mixing client updates before aggregation, CARAT evaluates updates through hidden task utility.

Trust-based and reference-data defenses. Trust-based defenses inject additional server knowledge into the aggregation process. FLTrust uses a small trusted server dataset to compute a trust anchor and reweight clients by their alignment with that anchor [11]. Other methods use client history or representation-level diagnostics rather than explicit trusted data; examples include FoolsGold, FLAME, DeepSight, and FLDetector [12–15]. CARAT is most closely related to this family because it also relies on server-side labeled reference data. The methodological difference is that CARAT neither compresses trusted information into a single global direction nor uses the trusted pool merely for post-hoc inspection. Instead, it expands the trusted pool into multiple hidden micro-tasks, yielding a more semantically resolved notion of client utility.

Adaptive poisoning attacks. Modern poisoning attacks are explicitly optimized against robust aggregators. FangAttack, MinMax, MinSum, and ALIE are representative examples [2, 3, 16]. On the backdoor side, model replacement, edge-case backdoors, and durable attacks such as Neurotoxin show that malicious updates can remain highly effective even when they are tuned to preserve outward plausibility [4–6]. This trend matters because it reduces the value of one-dimensional defenses. If an attacker only needs to satisfy a single criterion, such as geometric centrality or bounded norm, adaptation becomes easier.

Backdoor-specific defenses. Several recent defenses explicitly target backdoor robustness in FL. CRFL combines clipping and smoothing to provide certified robustness guarantees under restricted perturbation regimes [17]. FLAME and DeepSight instead use clustering and model-inspection signals to filter or sanitize suspicious updates [13, 14]. CARAT is complementary to these methods: it uses trusted data, but deploys that data as a collection of hidden task probes rather than as a purely global trust direction or a post-hoc inspection signal. This design places CARAT between trust-based aggregation and task-aware validation.

Positioning of CARAT. The closest comparison points for CARAT are FLTrust, bucketing-based robust aggregation, and backdoor-oriented inspection defenses such as FLAME and DeepSight. Compared with FLTrust, CARAT uses trusted data to create *many* hidden validation tasks rather than a single trusted direction. Compared with bucketing, CARAT does not attempt to reduce heterogeneity by averaging clients before robust aggregation; instead, it asks whether each client update remains useful across hidden semantic slices of the task. Compared with FLAME and DeepSight, CARAT intervenes directly at aggregation time rather than primarily through clustering or post-hoc model inspection. In short, CARAT is not mainly a new distance rule or inspection heuristic, but a task-aware aggregation mechanism for the regime in which benign heterogeneity and adaptive poisoning interact.

8 Conclusion

CARAT combines three ideas in a single aggregation rule: hidden class-balanced task certificates, rank-space anomaly suppression, and capped-simplex weight optimization. Together, these components aim to distinguish benign diversity from malicious inconsistency more effectively than geometry-only or single-reference defenses. The current implementation and logs already support a concrete methodological claim and an initial empirical claim: in a logged CIFAR100/FangAttack run, CARAT suppresses attacker weights and substantially improves over Mean, albeit with higher runtime and without yet surpassing every strong baseline on that single seed. The remaining step is to complete the empirical case with clean-test and ASR evaluation, multi-seed reruns, non-i.i.d. settings, and code-aligned ablations.

References

[1] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics*, 2017.

[2] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. Local model poisoning attacks to Byzantine-robust federated learning. In *USENIX Security Symposium*, 2020.

[3] Virat Shejwalkar and Amir Houmansadr. Manipulating the Byzantine: Optimizing model poisoning attacks and defenses for federated learning. In *Network and Distributed System Security Symposium*, 2021.

[4] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. How to backdoor federated learning. In *International Conference on Artificial Intelligence and Statistics*, pages 2938–2948, 2020.

[5] Hongyi Wang, Kartik Sreenivasan, Shashank Rajput, Harit Vishwakarma, Saurabh Agarwal, Jy-yong Sohn, Kangwook Lee, and Dimitris Papailiopoulos. Attack of the tails: Yes, you really can backdoor federated learning. In *Advances in Neural Information Processing Systems*, 2020.

[6] Zhengming Zhang, Ashwinee Panda, Linyue Song, Yaoqing Yang, Michael Mahoney, Prateek Mittal, Ramchandran Kannan, and Joseph Gonzalez. Neurotoxin: Durable backdoors in federated learning. In *International Conference on Machine Learning*, pages 26429–26446, 2022.

[7] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems*, 2017.

[8] El Mahdi El Mhamdi, Rachid Guerraoui, and Sébastien Rouault. The hidden vulnerability of distributed learning in Byzantium. In *International Conference on Machine Learning*, 2018.

[9] Dong Yin, Yudong Chen, Ramchandran Kannan, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *International Conference on Machine Learning*, 2018.

[10] Sai Praneeth Karimireddy, Lie He, and Martin Jaggi. Byzantine-robust learning on heterogeneous datasets via bucketing. In *International Conference on Learning Representations*, 2022.

[11] Xiaoyu Cao, Minghong Fang, Jia Liu, Neil Zhenqiang Gong, and Yixin Gong. FLTrust: Byzantine-robust federated learning via trust bootstrapping. In *Network and Distributed System Security Symposium*, 2021.

[12] Clement Fung, Chris J. M. Yoon, and Ivan Beschastnikh. The limitations of federated learning in sybil settings. In *International Symposium on Research in Attacks, Intrusions and Defenses*, 2020.

[13] Thien Duc Nguyen, Phillip Rieger, Hieu Yalame, Hauke Möllering, Huili Chen Xu, Ryohei Myung, Swarun Kadhe, and Kang G. Shin. FLAME: Taming backdoors in federated learning. In *USENIX Security Symposium*, 2022.

[14] Phillip Rieger, Thien Duc Nguyen, Hauke Möllering, Hieu Yalame, Huili Chen Xu, Ryohei Myung, Swarun Kadhe, and Kang G. Shin. DeepSight: Mitigating backdoor attacks in federated learning through deep model inspection. In *Network and Distributed System Security Symposium*, 2022.

[15] Zaixi Zhang, Xiaoyu Xu, Jinyuan Jia, Matthew Jagielski, Gauri Joshi, Mingyi Hong, and Carl A. Gunter. FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022.

[16] Gilad Baruch, Moran Baruch, and Yoav Goldberg. A little is enough: Circumventing defenses for distributed learning. In *Advances in Neural Information Processing Systems*, 2019.

[17] Chulin Xie, Minghao Chen, Pin-Yu Chen, and Bo Li. CRFL: Certifiably robust federated learning against backdoor attacks. In *International Conference on Machine Learning*, pages 11372–11382, 2021.

A Theoretical Analysis

Proposition A.1 (Feasibility and support of the capped simplex)**.** *Let*

$$\mathcal{A}(\hat{\rho}) = \left\{ \alpha \in \mathbb{R}^n : \alpha_i \geq 0,\ \sum_{i=1}^{n} \alpha_i = 1,\ \alpha_i \leq \frac{1}{(1-\hat{\rho})n} \right\},$$

with $\hat{\rho} < 1$. Then $\mathcal{A}(\hat{\rho})$ is non-empty, and every $\alpha \in \mathcal{A}(\hat{\rho})$ satisfies

$$|\operatorname{supp}(\alpha)| \geq (1-\hat{\rho})n.$$

Proof sketch. Let $u = 1/((1-\hat{\rho})n)$. Since $\hat{\rho} < 1$, we have $u \geq 1/n$, so the uniform weight vector is feasible and the set is non-empty. For the support bound, if only k coordinates of α are positive, then

$$1 = \sum_{i=1}^{n} \alpha_i \leq ku = \frac{k}{(1-\hat{\rho})n},$$

which implies $k \geq (1-\hat{\rho})n$. Therefore CARAT can suppress suspicious clients, but cannot collapse the aggregate onto an arbitrarily small subset of participants. □

Proposition A.2 (Softmin bounds and gradient form)**.** *Let $s \in \mathbb{R}^T$ and define*

$$\operatorname*{softmin}_{\beta}(s) = -\frac{1}{\beta} \log \left(\frac{1}{T} \sum_{t=1}^{T} e^{-\beta s_t} \right).$$

Then

$$\min_t s_t \leq \operatorname*{softmin}_{\beta}(s) \leq \frac{1}{T} \sum_{t=1}^{T} s_t.$$

Moreover, if $z(\alpha) = C^\top \alpha$ and

$$\pi_t(\alpha) = \frac{\exp(-\beta z_t(\alpha))}{\sum_{\ell=1}^{T} \exp(-\beta z_\ell(\alpha))},$$

then

$$\nabla_\alpha \operatorname*{softmin}_{\beta}(C^\top \alpha) = C\pi(\alpha).$$

Proof sketch. The lower and upper bounds follow from standard log-sum-exp inequalities: the exponential average is bounded below by the largest exponential term and above by the arithmetic mean. Monotonicity in each coordinate is immediate. Differentiating the softmin expression with respect to z_t yields the Gibbs weights $\pi_t(\alpha)$, and the chain rule then gives $C\pi(\alpha)$. Hence the ascent direction is a convex combination of certificate columns, with more weight placed on tasks whose current aggregate utility is smaller. □

Proposition A.3 (Directional invariance of certificate scoring)**.** *Suppose two nonzero clipped updates satisfy $\bar{\Delta}_i = a\bar{\Delta}_j$ for some scalar $a > 0$. Then their normalized versions in Eq.* (3) *coincide, i.e.,*

$$\tilde{\Delta}_i = \tilde{\Delta}_j.$$

Consequently, their hidden-task certificates are identical:

$$c_{i,t} = c_{j,t} \qquad \textit{for all } t \in \{1, \ldots, T\}.$$

Proof sketch. Eq. (3) rescales every nonzero clipped update to the common radius q_r. Positive collinearity is therefore removed by normalization:

$$q_r \frac{\bar{\Delta}_i}{\|\bar{\Delta}_i\|_2} = q_r \frac{a\bar{\Delta}_j}{\|a\bar{\Delta}_j\|_2} = q_r \frac{\bar{\Delta}_j}{\|\bar{\Delta}_j\|_2}.$$

Substituting the resulting equality into Eq. (4) gives identical task scores. Thus, after clipping, the certificate stage depends on direction rather than residual norm magnitude. □

Proposition A.4 (Temporal prior and projected-iterate invariants)**.** *Define*

$$f(\alpha) = \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\text{prev}}\|_2^2.$$

If $\alpha, \alpha' \in \mathcal{A}(\hat{\rho})$ satisfy $C^\top \alpha = C^\top \alpha'$ and $r^\top \alpha = r^\top \alpha'$, then

$$f(\alpha) - f(\alpha') = -\lambda_p \left(\|\alpha - \alpha^{\text{prev}}\|_2^2 - \|\alpha' - \alpha^{\text{prev}}\|_2^2 \right).$$

In addition, under projected gradient ascent

$$\alpha^{(k+1)} = \Pi_{\mathcal{A}(\hat{\rho})} \left(\alpha^{(k)} + \eta \nabla f(\alpha^{(k)}) \right),$$

every iterate remains in $\mathcal{A}(\hat{\rho})$.

Proof sketch. The first statement follows by direct substitution: if the certificate and rank terms match, the only remaining difference in objective value is the quadratic prior term. CARAT therefore prefers, among utility-equivalent feasible weights, the solution closer to α^{prev}. The second statement is immediate from the projection operator. Hence the optimizer preserves three round invariants throughout: $\alpha_i \geq 0$, $\sum_i \alpha_i = 1$, and $\alpha_i \leq 1/((1-\hat{\rho})n)$. □

Proposition A.5 (Per-round complexity)**.** *Let n be the number of clients, T the number of hidden tasks, m the number of sampled coordinates for rank scoring, S the number of rank subsamples, and K the number of projected-gradient steps. Ignoring constant factors from batch size and model width, the extra defense-side work of CARAT scales as*

$$\textit{probe-model forward passes} + O(Snm + KnT).$$

Proof sketch. CARAT performs one baseline probe evaluation plus n candidate probe evaluations over the T hidden tasks, then computes S rank summaries over m coordinates, and finally executes K projected optimization steps over n weights and T task scores. In the current implementation, the probe-model forward passes dominate wall-clock time, while the rank and optimization stages are lower-order terms controlled by m, S, and K. □

Remark on assumptions. The theoretical object D^{ref} in the main text should be read as the probe pool *after* subsampling. The code, however, obtains that pool by sampling from the full server-visible training dataset. Any final formal discussion should keep this distinction explicit, because it materially strengthens the server-side assumption relative to a fully data-free defense.

B Algorithm

Inputs. Current global model w_r, client messages $\{u_i^r\}_{i=1}^n$, previous-round weights α^{prev}, server-visible training dataset, and CARAT hyperparameters.

Persistent state. CARAT maintains two pieces of state across rounds: a balanced probe pool sampled at initialization from the server-visible training dataset, and the previous-round weight vector α^{prev}, which serves as the temporal prior in the current round.

Initialization. At the first round, the implementation initializes α^{prev} to the uniform vector. This makes the initial round unbiased across clients while still allowing later rounds to exploit temporal consistency once CARAT has observed several optimization trajectories.

Procedure.

1. Convert client messages to vector-form deltas Δ_i^r using the common update-preparation utility.
2. Compute client norms, choose a clipping threshold with the configured MAD or percentile rule, and form clipped deltas $\bar{\Delta}_i^r$.
3. Choose an evaluation radius from the configured quantile of positive clipped norms and normalize clipped deltas to obtain $\tilde{\Delta}_i^r$.

4. Sample class-balanced hidden tasks from the fixed probe pool for round r; if strict balanced sampling fails, fall back to replacement and, if necessary, to a random subset rather than aborting the round.
5. Load the current global weights into the server-side probe model, evaluate baseline hidden-task losses, and compute certificate scores $c_{i,t} = \ell_t(w_r) - \ell_t(w_r + \tilde{\Delta}_i^r)$ for all clients and tasks.
6. Robustly standardize certificate columns, compute rank-space penalties from coordinate subsamples of $\bar{\Delta}_i^r$, and retain only positive standardized rank deviations.
7. Resolve $\hat{\rho}$, initialize weights from α^{prev}, and optimize

$$\max_{\alpha \in \mathcal{A}(\hat{\rho})} \operatorname*{softmin}_{\beta}(C^\top \alpha) - \lambda_r r^\top \alpha - \lambda_p \|\alpha - \alpha^{\text{prev}}\|_2^2$$

   by projected gradient ascent on the capped simplex.
8. Update α^{prev} with the optimized solution.
9. Aggregate the clipped deltas $\bar{\Delta}_i^r$ with the optimized weights and return the resulting global step.

Round invariants and failure handling. Two implementation choices are important for robustness. First, the optimizer never leaves the capped simplex, so every intermediate and final weight vector remains feasible. Second, probe-task construction is fail-soft rather than fail-stop: if strict class-balanced sampling is impossible, the code falls back to replacement or to a random subset, ensuring that the round can still proceed with a well-defined certificate matrix.

Outputs. Aggregated server update, updated temporal prior α^{prev}, and diagnostic logs including learned weights, task scores, softmin focus, rank penalties, evaluation radius, clipping threshold, and the identities of the hidden probe tasks used in the current round.

C Additional Experiments

This section summarizes the current verified experimental scope together with the next empirical extensions already supported by the repository structure.

Currently verified run. The strongest directly verified CARAT run in the repository uses FedAvg, CIFAR100, ResNet-18, an i.i.d. partition, 20 clients, 4 attackers, FangAttack, 300 rounds, learning rate 0.05, local epochs 4, and `evaluate=false`. The main text already reports the final training metrics and runtime of this run together with the corresponding Mean and TriGuardFL baselines.

Supported benchmark matrix. The committed `cifar100.yaml` spec already supports i.i.d. and non-i.i.d. splits with Dirichlet $\alpha \in \{0.1, 0.5, 1.0\}$, attacker fractions $\{0.2, 0.3, 0.4\}$, and untargeted attacks {NoAttack, MinMax, MinSum, ALIE, FangAttack}. The next revision should add CARAT into that matrix rather than relying on one-off runs.

Next additions.

1. Add CARAT to the main batch experiment specs and rerun the CIFAR100 benchmark with `evaluate=true` so that clean test accuracy and ASR are recorded every round.
2. Extend the verified runs from the current i.i.d. FangAttack setting to the non-i.i.d. Dirichlet settings already present in the committed benchmark spec.
3. Add at least one backdoor suite, such as BadNets or Neurotoxin, so that the paper reports both clean utility and ASR.
4. Run the config-supported ablations in Table 2, prioritizing the number of hidden tasks, the rank and prior weights, and the clipping controls.
5. Report multi-seed means and variances, plus defense runtime overhead, before making any strong comparative claim against other robust aggregators.